```
#Monalika_Kubde - 23070521094
# ============================================================
# Generative Adversarial Network (GAN) on MNIST
# ============================================================
#
# Problem Statement:
# Develop a GAN model to generate synthetic handwritten digit
# images.
#
# Activities:
# 1. Implement Generator and Discriminator
# 2. Train GAN model
# 3. Visualize generated images
# ============================================================


# ------------------------------------------------------------
# 1. Import Libraries
# ------------------------------------------------------------

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Dense,
    LeakyReLU,
    BatchNormalization,
    Reshape,
    Flatten,
    Dropout
)
from tensorflow.keras.optimizers import Adam


# ------------------------------------------------------------
# 2. Load MNIST Dataset
# ------------------------------------------------------------

(x_train, _), (_, _) = mnist.load_data()

print("Original dataset shape:", x_train.shape)


# ------------------------------------------------------------
# 3. Preprocess Images
# ------------------------------------------------------------

# Convert pixel values from [0, 255] to [-1, 1]
# This matches the tanh activation used by the Generator.

x_train = x_train.astype("float32")

x_train = (x_train - 127.5) / 127.5

# Add channel dimension
# Shape changes from:
# (60000, 28, 28)
# to:
# (60000, 28, 28, 1)

x_train = np.expand_dims(x_train, axis=-1)

print("Preprocessed dataset shape:", x_train.shape)


# ------------------------------------------------------------
# 4. Parameters
# ------------------------------------------------------------

IMG_ROWS = 28
IMG_COLS = 28
CHANNELS = 1
```

```
IMG_SHAPE = (IMG_ROWS, IMG_COLS, CHANNELS)

LATENT_DIM = 100

BATCH_SIZE = 128
EPOCHS = 30


# ------------------------------------------------------------
# 5. Build Generator
# ------------------------------------------------------------

generator = Sequential(
    name="Generator"
)

generator.add(
    Dense(
        256,
        input_dim=LATENT_DIM
    )
)

generator.add(
    LeakyReLU(negative_slope=0.2)
)

generator.add(
    BatchNormalization()
)

generator.add(
    Dense(512)
)

generator.add(
    LeakyReLU(negative_slope=0.2)
)

generator.add(
    BatchNormalization()
)

generator.add(
    Dense(1024)
)

generator.add(
    LeakyReLU(negative_slope=0.2)
)

generator.add(
    BatchNormalization()
)

# Generate 28 x 28 x 1 = 784 pixels
generator.add(
    Dense(
        28 * 28,
        activation="tanh"
    )
)

# Convert 784 values into 28 x 28 x 1 image
generator.add(
    Reshape(IMG_SHAPE)
)


print("\n==============================")
print("GENERATOR")
print("==============================")

generator.summary()


# ------------------------------------------------------------
```

```
# 6. Build Discriminator
# ------------------------------------------------------------

discriminator = Sequential(
    name="Discriminator"
)

# Convert image into a one-dimensional vector
discriminator.add(
    Flatten(
        input_shape=IMG_SHAPE
    )
)

discriminator.add(
    Dense(512)
)

discriminator.add(
    LeakyReLU(negative_slope=0.2)
)

discriminator.add(
    Dropout(0.3)
)

discriminator.add(
    Dense(256)
)

discriminator.add(
    LeakyReLU(negative_slope=0.2)
)

discriminator.add(
    Dropout(0.3)
)

# Output probability:
# 0 = Fake
# 1 = Real

discriminator.add(
    Dense(
        1,
        activation="sigmoid"
    )
)


print("\n==============================")
print("DISCRIMINATOR")
print("==============================")

discriminator.summary()


# ------------------------------------------------------------
# 7. Compile Discriminator
# ------------------------------------------------------------

optimizer = Adam(
    learning_rate=0.0002,
    beta_1=0.5
)

discriminator.compile(
    optimizer=optimizer,
    loss="binary_crossentropy",
    metrics=["accuracy"]
)


# ------------------------------------------------------------
# 8. Build Complete GAN
# ------------------------------------------------------------
```

```
# Freeze discriminator while training the combined GAN.
# This means that when the Generator is trained,
# only Generator weights are updated.

discriminator.trainable = False


gan = Sequential(
    [
        generator,
        discriminator
    ],
    name="GAN"
)


gan.compile(
    optimizer=Adam(
        learning_rate=0.0002,
        beta_1=0.5
    ),
    loss="binary_crossentropy"
)


print("\n==============================")
print("COMPLETE GAN")
print("==============================")

gan.summary()


# ------------------------------------------------------------
# 9. Training Function
# ------------------------------------------------------------

def train_gan():

    # Store losses for visualization
    discriminator_losses = []
    generator_losses = []

    batches_per_epoch = x_train.shape[0] // BATCH_SIZE

    for epoch in range(EPOCHS):

        d_loss_epoch = []
        g_loss_epoch = []

        for batch in range(batches_per_epoch):

            # ------------------------------------------------
            # Step 1: Select a batch of real images
            # ------------------------------------------------

            real_images = x_train[
                np.random.randint(
                    0,
                    x_train.shape[0],
                    BATCH_SIZE
                )
            ]


            # ------------------------------------------------
            # Step 2: Generate random noise
            # ------------------------------------------------

            noise = np.random.normal(
                0,
                1,
                (BATCH_SIZE, LATENT_DIM)
            )


            # ------------------------------------------------
            # Step 3: Generate fake images
            # ------------------------------------------------
```

```
#

fake_images = generator.predict(
    noise,
    verbose=0
)


# ------------------------------------------------
# Step 4: Train Discriminator
# ------------------------------------------------

# Real images have label 1
real_labels = np.ones(
    (BATCH_SIZE, 1)
)

# Fake images have label 0
fake_labels = np.zeros(
    (BATCH_SIZE, 1)
)


# Train on real images
d_loss_real = discriminator.train_on_batch(
    real_images,
    real_labels
)


# Train on fake images
d_loss_fake = discriminator.train_on_batch(
    fake_images,
    fake_labels
)


# Average discriminator loss
d_loss = (
    np.array(d_loss_real) +
    np.array(d_loss_fake)
) / 2


# ------------------------------------------------
# Step 5: Train Generator
# ------------------------------------------------

# Generate new random noise
noise = np.random.normal(
    0,
    1,
    (BATCH_SIZE, LATENT_DIM)
)


# We want the Discriminator to think
# generated images are REAL.
#
# Therefore labels are 1.

valid_labels = np.ones(
    (BATCH_SIZE, 1)
)


# Train complete GAN
g_loss = gan.train_on_batch(
    noise,
    valid_labels
)


# Store losses
d_loss_epoch.append(d_loss[0])
g_loss_epoch.append(
    g_loss[0] if isinstance(g_loss, (list, tuple))
    else g_loss
```

```
            )


        # ----------------------------------------------------
        # Calculate Average Losses
        # ----------------------------------------------------

        avg_d_loss = np.mean(
            d_loss_epoch
        )

        avg_g_loss = np.mean(
            g_loss_epoch
        )

        discriminator_losses.append(
            avg_d_loss
        )

        generator_losses.append(
            avg_g_loss
        )


        # ----------------------------------------------------
        # Display Progress
        # ----------------------------------------------------

        print(
            f"Epoch {epoch + 1}/{EPOCHS} "
            f"- D Loss: {avg_d_loss:.4f} "
            f"- G Loss: {avg_g_loss:.4f}"
        )


    return (
        discriminator_losses,
        generator_losses
    )


# ------------------------------------------------------------
# 10. Train GAN
# ------------------------------------------------------------

print("\n==============================")
print("STARTING GAN TRAINING")
print("==============================")

d_losses, g_losses = train_gan()


# ------------------------------------------------------------
# 11. Generate New Images
# ------------------------------------------------------------

num_images = 25

noise = np.random.normal(
    0,
    1,
    (num_images, LATENT_DIM)
)

generated_images = generator.predict(
    noise,
    verbose=0
)


# ------------------------------------------------------------
# 12. Convert Images from [-1, 1] to [0, 1]
# ------------------------------------------------------------

generated_images = (
    generated_images + 1
) / 2.0
```

```
# ------------------------------------------------------------
# 13. Visualize Generated Images
# ------------------------------------------------------------

plt.figure(
    figsize=(10, 10)
)

for i in range(num_images):

    plt.subplot(5, 5, i + 1)

    plt.imshow(
        generated_images[i].reshape(
            28,
            28
        ),
        cmap="gray"
    )

    plt.axis("off")


plt.suptitle(
    "Generated Handwritten Digits using GAN",
    fontsize=16
)

plt.tight_layout()

plt.show()


# ------------------------------------------------------------
# 14. Plot Generator and Discriminator Loss
# ------------------------------------------------------------

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    d_losses,
    label="Discriminator Loss"
)

plt.plot(
    g_losses,
    label="Generator Loss"
)

plt.title(
    "GAN Training Loss"
)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()


# ------------------------------------------------------------
# 15. Save Generator
# ------------------------------------------------------------

generator.save(
    "mnist_gan_generator.keras"
)

print(
    "\nGenerator saved as "
```

```
    "'mnist_gan_generator.keras'"
)


# ------------------------------------------------------------
# 16. Generate One New Digit
# ------------------------------------------------------------

random_noise = np.random.normal(
    0,
    1,
    (1, LATENT_DIM)
)

new_digit = generator.predict(
    random_noise,
    verbose=0
)

new_digit = (
    new_digit + 1
) / 2.0


plt.figure(figsize=(4, 4))

plt.imshow(
    new_digit[0].reshape(28, 28),
    cmap="gray"
)

plt.title("New Synthetic Digit")

plt.axis("off")

plt.show()
```

```
Original dataset shape: (60000, 28, 28)
Preprocessed dataset shape: (60000, 28, 28, 1)

==============================
GENERATOR
==============================
```

Model: "Generator"

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 256)	25,856
leaky_re_lu_10 (LeakyReLU)	(None, 256)	0
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dense_15 (Dense)	(None, 512)	131,584
leaky_re_lu_11 (LeakyReLU)	(None, 512)	0
batch_normalization_7 (BatchNormalization)	(None, 512)	2,048
dense_16 (Dense)	(None, 1024)	525,312
leaky_re_lu_12 (LeakyReLU)	(None, 1024)	0
batch_normalization_8 (BatchNormalization)	(None, 1024)	4,096
dense_17 (Dense)	(None, 784)	803,600
reshape_2 (Reshape)	(None, 28, 28, 1)	0

Total params: 1,493,520 (5.70 MB)
Trainable params: 1,489,936 (5.68 MB)
Non-trainable params: 3,584 (14.00 KB)

```
==============================
DISCRIMINATOR
==============================
```

Model: "Discriminator"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_18 (Dense)	(None, 512)	401,920
leaky_re_lu_13 (LeakyReLU)	(None, 512)	0
dropout_4 (Dropout)	(None, 512)	0
dense_19 (Dense)	(None, 256)	131,328
leaky_re_lu_14 (LeakyReLU)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_20 (Dense)	(None, 1)	257

Total params: 533,505 (2.04 MB)
Trainable params: 533,505 (2.04 MB)
Non-trainable params: 0 (0.00 B)